# AIML ZG519 – Natural Language Processing Applications

# Assignment: Sentiment Analysis – A Comparative Study

**Session Reference:** Session 6 (Sentiment Analysis) — builds on Sessions 3–5 (RAG, QA, Conversational AI)

**Group No.:** 86 **Problem Statement:** PS4 — Restaurant / Food-Delivery Reviews **Group Members & Contributions:**

| # | Name | BITS ID | Contribution Areas |
|---|---|---|---|
| 1 | **Prajwal Shetty K P** | 2025AE05434 | ① Dataset load & stratified 10k sample (Yelp Polarity) ② Exploratory data analysis ③ Part A — light + heavy preprocessing pipelines ④ Part B — classical ML: TF-IDF + Naive Bayes & GridSearchCV-tuned Logistic Regression |
| 2 | **Neha Yadav** | 2025AE05463 | Part C — Deep Learning: embedding + BiLSTM classifier, train/validation curves, test evaluation |
| 3 | **Poornima Prakash Patil** | 2025AE05411 | Part D — Transformer fine-tuning: DistilBERT via HuggingFace Trainer, metrics, confusion matrix & inference latency |
| 4 | **Narumalla Pavan Kumar** | 2025AE05457 | ① Part E — GenAI/LLM prompting (zero-shot & few-shot) ② Part F — consolidated metrics table, comparison charts, robustness stress-test & written summary |

---

## Objective

In this assignment you will build **four different sentiment analysis pipelines** on the same dataset — classical Machine Learning, Deep Learning, a fine-tuned Transformer, and a Generative AI / LLM prompting approach — and compare them on accuracy, robustness, interpretability, latency and cost.

**Total Marks: 10**

Fill in every cell marked `# TODO`. Do not delete the markdown instructions — they are part of your submission and will be used for evaluation. Answer the **reflection questions** at the end of each part in the markdown cell provided.

## Learning Outcomes

- LO2: Hands-on experience with major NLP technologies and tools
- LO3: Apply NLP techniques (sentiment analysis) using state-of-the-art Gen AI techniques
- LO4: Sharpen programming skills for NLP applications

# 0. Environment Setup

Run this cell first. If you are on Google Colab, uncomment the `pip install` line.

In [1]:
```
# If starting from a clean environment (Google Colab / fresh venv), install depende
# !pip install -q scikit-learn nltk datasets wordcloud pandas pyarrow matplotlib se
# Parts C–E additionally need:  tensorflow  transformers  torch  openai

import os, re, time, json, random, warnings
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

RANDOM_STATE = 42
random.seed(RANDOM_STATE)
np.random.seed(RANDOM_STATE)

warnings.filterwarnings("ignore")
sns.set_style("whitegrid")
pd.set_option("display.max_colwidth", 120)
%matplotlib inline

DATA_DIR = Path("data")
DATA_DIR.mkdir(exist_ok=True)   # shared artefacts for Members 2–4 land here
```

# 1. Dataset — Your Assigned Problem Statement

Your group has been assigned ONE problem statement (PS1–PS7) in the **Assignment Brief**, based on your group number. Fill in the table below with your group's details before doing anything else — the TA will cross-check this against the group allocation list.

| Field | Your Group's Value |
|---|---|
| Group Number | **86** |
| Problem Statement ID | **PS4** |
| Domain | **Restaurant / Food-Delivery** (Food & Hospitality) |
| Dataset used | **Yelp Polarity** — HuggingFace `fancyzhx/yelp_polarity` (canonical mirror of `yelp_polarity`): 560,000 train + 38,000 test reviews, binary label **0 = Negative / 1 = Positive** |

| Field | Your Group's Value |
|---|---|
| Aspect focus (for Parts F & G) | **taste / quality, delivery time, hygiene, packaging** |

**Business scenario (from the brief):** a food-delivery platform wants to detect dissatisfaction signals for restaurant-partner quality control — so recall on the **Negative** class is the metric that matters most in deployment.

**Sampling:** Yelp Polarity is far larger than needed, so we draw a **stratified 10,000-row sample** (5,000 per class) and make a single 80/20 train/test split. That exact sample is persisted to `data/ps4_sample_{train,test}.csv` and reused unchanged by Parts B–E so the four-way comparison is fair.

| PS | Domain | Suggested Dataset | Aspect Focus |
|---|---|---|---|
| PS1 | E-Commerce Product Reviews | `amazon_polarity` (one category) or Flipkart Reviews (Kaggle) | price, delivery, quality, packaging |
| PS2 | Movie / OTT Reviews | `imdb` or Rotten Tomatoes | acting, plot, cinematography, pacing |
| PS3 | Airline & Travel | Twitter US Airline Sentiment (Kaggle) | delay, crew behaviour, baggage, comfort |
| PS4 | Restaurant / Food-Delivery | `yelp_polarity` or Zomato Reviews (Kaggle) | taste, delivery time, hygiene, packaging |
| PS5 | Healthcare: Patient Drug Reviews | UCI Drug Review Dataset (Kaggle) | effectiveness, side effects, ease of use |
| PS6 | Financial News & Market Sentiment | `financial_phrasebank` or Twitter Financial News Sentiment | earnings, macro risk, bullish/bearish tone |
| PS7 | Mobile App Store Reviews | Google Play Store App Reviews (Kaggle) or `app_reviews` (HuggingFace) | app crashes/bugs, UI/UX, feature requests, pricing |

In [2]:
```
# ----------------------------------------------------------------------------
# PS4 dataset: Yelp Polarity (Restaurant / Food-Delivery reviews, binary sentiment)
# The brief lists load_dataset("yelp_polarity"); that bare alias is broken on
# `datasets` >= 3, so we load the identical canonical repo `fancyzhx/yelp_polarity`
# ----------------------------------------------------------------------------
from datasets import load_dataset
from sklearn.model_selection import train_test_split

SAMPLE_SIZE = 10_000          # stratified sample reused across Parts B–E (see bri
LABEL_NAMES = {0: "Negative", 1: "Positive"}

raw = load_dataset("fancyzhx/yelp_polarity")
full = pd.concat([pd.DataFrame(raw["train"]), pd.DataFrame(raw["test"])], ignore_in
full["label"] = full["label"].astype(int)
full["text"] = full["text"].astype(str).str.replace(r"\\n", " ", regex=True)
full["sentiment"] = full["label"].map(LABEL_NAMES)
print(f"Full corpus : {len(full):,} rows | class counts: {full['sentiment'].value_c
```

```
# ---- stratified down-sample: equal rows per class, then shuffle ----
per_class = SAMPLE_SIZE // full["label"].nunique()
sample_df = (full.groupby("label", group_keys=False)
                 .sample(n=per_class, random_state=RANDOM_STATE)
                 .sample(frac=1, random_state=RANDOM_STATE)
                 .reset_index(drop=True))

# ---- ONE train/test split, reused by every downstream approach ----
train_df, test_df = train_test_split(
    sample_df, test_size=0.20, stratify=sample_df["label"], random_state=RANDOM_STA
train_df = train_df.reset_index(drop=True)
test_df = test_df.reset_index(drop=True)

print(f"Sample      : {len(sample_df):,} rows (stratified, {per_class} per class)")
print(f"Train / Test: {len(train_df):,} / {len(test_df):,}")
print("Train balance:", train_df['sentiment'].value_counts(normalize=True).round(3)
print("Test  balance:", test_df['sentiment'].value_counts(normalize=True).round(3).
train_df[["text", "label", "sentiment"]].head(3)
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOK
EN to enable higher rate limits and faster downloads.
Full corpus : 598,000 rows | class counts: {'Negative': 299000, 'Positive': 299000}
Sample      : 10,000 rows (stratified, 5000 per class)
Train / Test: 8,000 / 2,000
Train balance: {'Negative': 0.5, 'Positive': 0.5}
Test  balance: {'Negative': 0.5, 'Positive': 0.5}
```

Out[2]:

| | text | label | sentiment |
|---|---|---|---|
| **0** | I left a glowing review for your closed location... I still stand by that review but the last two times I have been ... | 0 | Negative |
| **1** | I love coming here after the bars. It is open late and their Greek fries are to die for!!! I've been coming here for... | 1 | Positive |
| **2** | Drinks are like juice. Free cover but drinks will cost you. Shots served in warm shot glasses. | 0 | Negative |

## 1.1 Exploratory Data Analysis (EDA)

Produce at minimum:

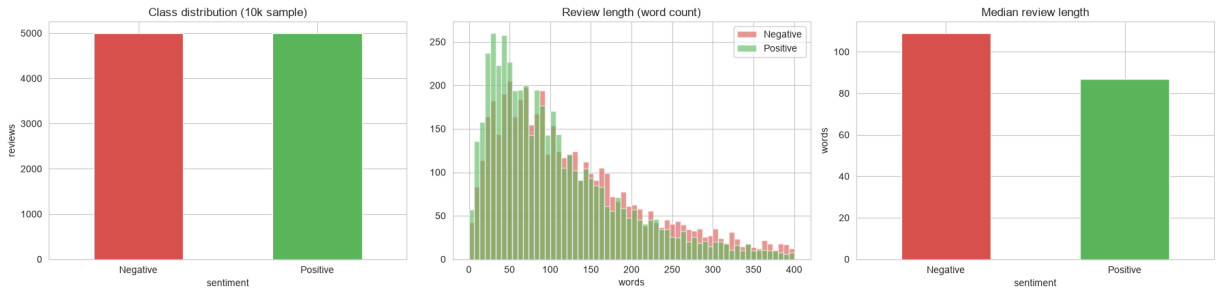
- Class distribution plot
- Review-length distribution (word count histogram)
- Word clouds or top-N frequent tokens per class

**Reflection Q1:** Is the dataset balanced? How might class imbalance affect model choice and evaluation metric selection later in this notebook?

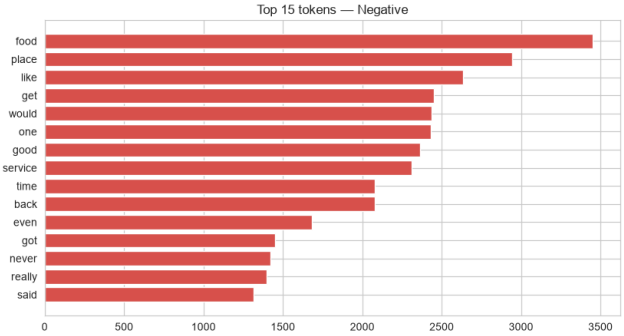
In [3]:

```
from collections import Counter
from wordcloud import WordCloud
import nltk
nltk.download("stopwords", quiet=True)
```

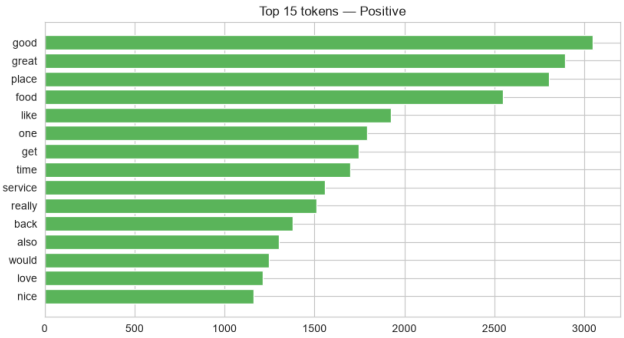
```
from nltk.corpus import stopwords
EN_STOP = set(stopwords.words("english"))

NEG, POS = "#d9534f", "#5cb85c"
sample_df["n_words"] = sample_df["text"].str.split().str.len()

# ---- (1) class balance + review-length distribution --------------------------
fig, ax = plt.subplots(1, 3, figsize=(18, 4.4))
sample_df["sentiment"].value_counts().reindex(["Negative", "Positive"]).plot(
    kind="bar", ax=ax[0], color=[NEG, POS]); ax[0].set_title("Class distribution (1
ax[0].set_ylabel("reviews"); ax[0].tick_params(axis="x", rotation=0)

for lbl, c in [("Negative", NEG), ("Positive", POS)]:
    ax[1].hist(sample_df.loc[sample_df.sentiment == lbl, "n_words"], bins=60,
               range=(0, 400), alpha=.6, label=lbl, color=c)
ax[1].set_title("Review length (word count)"); ax[1].set_xlabel("words"); ax[1].leg

sample_df.groupby("sentiment")["n_words"].median().reindex(["Negative", "Positive"]
    kind="bar", ax=ax[2], color=[NEG, POS]); ax[2].set_title("Median review length"
ax[2].set_ylabel("words"); ax[2].tick_params(axis="x", rotation=0)
plt.tight_layout(); plt.show()

print(sample_df.groupby("sentiment")["n_words"].describe()[["mean", "50%", "min", "

# ---- (2) top tokens + word clouds per class ----------------------------------
def top_tokens(texts, k=15):
    c = Counter()
    for t in texts:
        c.update(w for w in re.findall(r"[a-z']{3,}", t.lower()) if w not in EN_STO
    return c.most_common(k)

fig, ax = plt.subplots(2, 2, figsize=(16, 9))
for row, (lbl, col) in enumerate([("Negative", NEG), ("Positive", POS)]):
    txt = sample_df.loc[sample_df.sentiment == lbl, "text"]
    words, counts = zip(*top_tokens(txt))
    y = range(len(words))
    ax[row, 0].barh(list(y)[::-1], counts, color=col)
    ax[row, 0].set_yticks(list(y)[::-1]); ax[row, 0].set_yticklabels(words)
    ax[row, 0].set_title(f"Top 15 tokens — {lbl}")
    wc = WordCloud(width=760, height=380, background_color="white",
                   stopwords=EN_STOP, collocations=False).generate(
                   " ".join(txt.sample(min(1500, len(txt)), random_state=RANDOM_STA
    ax[row, 1].imshow(wc); ax[row, 1].axis("off"); ax[row, 1].set_title(f"Word clou
plt.tight_layout(); plt.show()
```

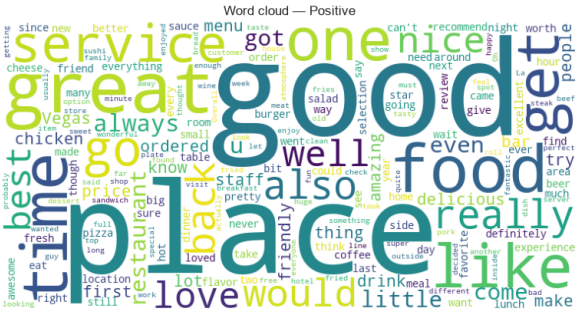


```
           mean    50%  min     max
sentiment
Negative   148.8  109.0  1.0  1002.0
Positive   117.5   87.0  1.0   928.0
```








*Your answer to Reflection Q1:*

**The dataset is balanced.** Yelp Polarity is 50/50 Negative/Positive by construction, and our stratified sample keeps that exactly (5,000 / 5,000; train and test both ≈ 50.0 % per class). Consequences for the rest of the notebook:

- **Accuracy is a fair headline metric** here — a majority-class baseline only reaches ~50 %, so there is no imbalance trap inflating it. We still report **weighted Precision / Recall / F1 and confusion matrices** because the deployment goal (flagging dissatisfaction for restaurant-partner quality control) makes **Negative-class recall** the number that really matters, and per-class metrics expose that where accuracy alone would hide it.
- **No resampling / class-weighting is needed.** If we later swapped in a naturally skewed source (e.g. Zomato reviews skew positive), we would switch the headline metric to **macro-F1** and add `class_weight="balanced"` — the metric set we already report would surface the problem.
- **Length is mildly class-dependent** — negative reviews run a little longer (people justify complaints). TF-IDF's `sublinear_tf` / L2 norm absorbs this for classical ML; the neural models handle it via a fixed `max_len` with padding/truncation.

# 2. Part A — Text Preprocessing (1 mark)

Implement a reusable `clean_text()` function. Keep two versions of the text:

- `text_classical` : heavily cleaned (lowercase, punctuation/stopword removal, lemmatization) — for the classical ML pipeline (Part 3).
- `text_neural` : lightly cleaned (lowercase, HTML/URL removal only) — for DL/Transformer/LLM pipelines (Parts 4–5), since those models learn their own representations and over-cleaning can hurt them.

In [4]:
```
import nltk
for pkg in ("stopwords", "wordnet", "omw-1.4"):
    nltk.download(pkg, quiet=True)
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

lemmatizer = WordNetLemmatizer()
# Negation words flip sentiment and a bag-of-words model has no other way to see th
# so we KEEP them instead of dropping them as stop-words.
NEGATION_KEEP = {"no", "not", "nor", "never", "cannot", "without", "hardly", "barel
STOP = set(stopwords.words("english")) - NEGATION_KEEP

_URL  = re.compile(r"http\S+|www\.\S+")
_HTML = re.compile(r"<.*?>")
_NNT  = re.compile(r"n['\u2019]t\b")


def clean_light(text: str) -> str:
    """Minimal cleaning — for DL / Transformer / LLM pipelines (Parts C–E)."""
    text = _HTML.sub(" ", str(text))
    text = _URL.sub(" ", text)
    text = text.replace("\\n", " ").replace("\n", " ")
    return re.sub(r"\s+", " ", text).strip().lower()


def clean_heavy(text: str) -> str:
    """Aggressive cleaning — for the TF-IDF / bag-of-words classical pipeline (Part
    text = clean_light(text)
    text = _NNT.sub(" not", text)                 # don't -> do not  (protect negat
    text = re.sub(r"[^a-z\s]", " ", text)          # drop digits & punctuation
    tokens = [lemmatizer.lemmatize(tok) for tok in text.split()
              if (tok not in STOP and len(tok) > 2) or tok in NEGATION_KEEP]
    return " ".join(tokens)


_t = time.time()
for _df in (train_df, test_df, sample_df):
    _df["text_neural"] = _df["text"].apply(clean_light)
    _df["text_classical"] = _df["text"].apply(clean_heavy)
print(f"Preprocessed {len(train_df) + len(test_df):,} rows in {time.time() - _t:.1f

for col in ("text", "text_classical", "text_neural"):
    print(f"[{col}]\n  {train_df.loc[0, col][:230]}\n")

# ---- persist the SHARED sample so Members 2-4 train/evaluate on identical data --
_keep = ["text", "text_neural", "text_classical", "label", "sentiment"]
train_df[_keep].to_csv(DATA_DIR / "ps4_sample_train.csv", index=False)
test_df[_keep].to_csv(DATA_DIR / "ps4_sample_test.csv", index=False)
```

```
print("Saved shared sample ->",
      f"{DATA_DIR/'ps4_sample_train.csv'} ({len(train_df)} rows),",
      f"{DATA_DIR/'ps4_sample_test.csv'} ({len(test_df)} rows)")
```

```
Preprocessed 10,000 rows in 5.5s

[text]
  I left a glowing review for your closed location... I still stand by that review b
ut the last two times I have been into this location the Ribs have been completely D
RY and OVERCOOKED!   If it were once I would say that it was an

[text_classical]
  left glowing review closed location still stand review last two time location rib
completely dry overcooked would say night two time row think need examine overcookin
g meat bbq tender juicy not dry hard hate leave bad review manag

[text_neural]
  i left a glowing review for your closed location... i still stand by that review b
ut the last two times i have been into this location the ribs have been completely d
ry and overcooked! if it were once i would say that it was an of

Saved shared sample -> data\ps4_sample_train.csv (8000 rows), data\ps4_sample_test.c
sv (2000 rows)
```

### Why the two pipelines differ — and why over-cleaning hurts neural / LLM models

`clean_heavy` **(Part B, TF-IDF):** lower-casing, punctuation & digit removal, stop-word removal, lemmatisation. A bag-of-words model scores every token independently, so collapsing the vocabulary to content-bearing lemmas raises each TF-IDF feature's signal-to-noise ratio and cuts dimensionality / overfitting. We deliberately **keep negation words** and rewrite `n't → not`, otherwise "*not tasty*" and "*tasty*" map to the same features.

`clean_light` **(Parts C–E):** HTML/URL stripping, whitespace and case normalisation only. Aggressive cleaning **degrades** these models:

- **Sub-word tokenisers already expect raw text.** DistilBERT's WordPiece vocabulary and an LLM's BPE were pre-trained on ordinary casing, punctuation and stop-words; stripped text is a train/serve mismatch against that pre-training distribution.
- **Sequence models rely on word order and function words.** "*The delivery was late but the food was great*" hinges on *but*, *was*, *the* and ordering — all destroyed by stop-word removal.
- **Punctuation and case carry sentiment.** "*Great. Just great.*" (sarcasm), "*!!!*", "*?*" and ALL-CAPS are informative to a transformer / LLM.
- **Lemmatisation discards intensity and tense** — "*worst*" → "*bad*", "*was refunded*" vs "*will refund*" — signals the attention layers would otherwise use.

# 3. Part B — Classical Machine Learning (2 marks)

Build **two** classical pipelines using `TfidfVectorizer` (or `CountVectorizer`) with:

1. **Naive Bayes** (`MultinomialNB`)
2. **One of:** Logistic Regression / Linear SVM

For each: report Accuracy, Precision, Recall, F1 (weighted), and a confusion matrix. Tune at least one hyperparameter (e.g. `ngram_range`, `max_features`, `C`) using `GridSearchCV` or `RandomizedSearchCV` and report the best configuration.

In [5]:
```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import (accuracy_score, precision_recall_fscore_support,
                             classification_report, confusion_matrix)

X_train, y_train = train_df["text_classical"], train_df["label"]
X_test,  y_test  = test_df["text_classical"],  test_df["label"]


def score(name, model, fit_time):
    pred = model.predict(X_test)
    acc = accuracy_score(y_test, pred)
    p, r, f1, _ = precision_recall_fscore_support(y_test, pred, average="weighted")
    print(f"\n=== {name} ===   (fit {fit_time:.1f}s)")
    print(classification_report(y_test, pred, target_names=["Negative", "Positive"]
    return dict(Model=name, Accuracy=acc, Precision=p, Recall=r, F1=f1,
                FitTime_s=round(fit_time, 2), Predictions=pred)


BASE_TFIDF = dict(ngram_range=(1, 2), min_df=3, max_features=30_000, sublinear_tf=T

# ---- Model 1: TF-IDF + Multinomial Naive Bayes ----
t = time.time()
nb = Pipeline([("tfidf", TfidfVectorizer(**BASE_TFIDF)),
               ("clf", MultinomialNB())]).fit(X_train, y_train)
res_nb = score("TF-IDF + MultinomialNB", nb, time.time() - t)

# ---- Model 2: TF-IDF + Logistic Regression (untuned baseline) ----
t = time.time()
lr = Pipeline([("tfidf", TfidfVectorizer(**BASE_TFIDF)),
               ("clf", LogisticRegression(max_iter=1000, C=1.0))]).fit(X_train, y_t
res_lr = score("TF-IDF + LogisticRegression", lr, time.time() - t)

# ---- Hyperparameter tuning: GridSearchCV over the Logistic Regression pipeline --
param_grid = {
    "tfidf__ngram_range":   [(1, 1), (1, 2)],
    "tfidf__min_df":        [2, 5],
    "tfidf__max_features":  [30_000, 50_000],
    "clf__C":               [0.5, 1.0, 2.0, 4.0],
}
t = time.time()
grid = GridSearchCV(
```

```
    Pipeline([("tfidf", TfidfVectorizer(sublinear_tf=True)),
              ("clf", LogisticRegression(max_iter=1000))]),
    param_grid, scoring="f1_weighted", cv=4, n_jobs=-1, verbose=1).fit(X_train, y_t
grid_time = time.time() - t
print(f"\nBest CV F1 = {grid.best_score_:.4f}")
print(f"Best params = {grid.best_params_}")
res_best = score("TF-IDF + LogReg (GridSearchCV-tuned)", grid.best_estimator_, grid

# ---- consolidated Part B metrics table ----
classical_results = (pd.DataFrame([res_nb, res_lr, res_best])
                       .drop(columns="Predictions")
                       .set_index("Model").round(4))

# hand-off to Part F (Member 4)
PART_F_METRICS = globals().get("PART_F_METRICS", {})
PART_F_METRICS["Naive Bayes"]  = dict(Accuracy=res_nb["Accuracy"],  F1=res_nb["F1"]
                                      TrainTime_s=res_nb["FitTime_s"])
PART_F_METRICS["LogReg (tuned)"] = dict(Accuracy=res_best["Accuracy"], F1=res_best[
                                        TrainTime_s=res_best["FitTime_s"])
classical_results
```

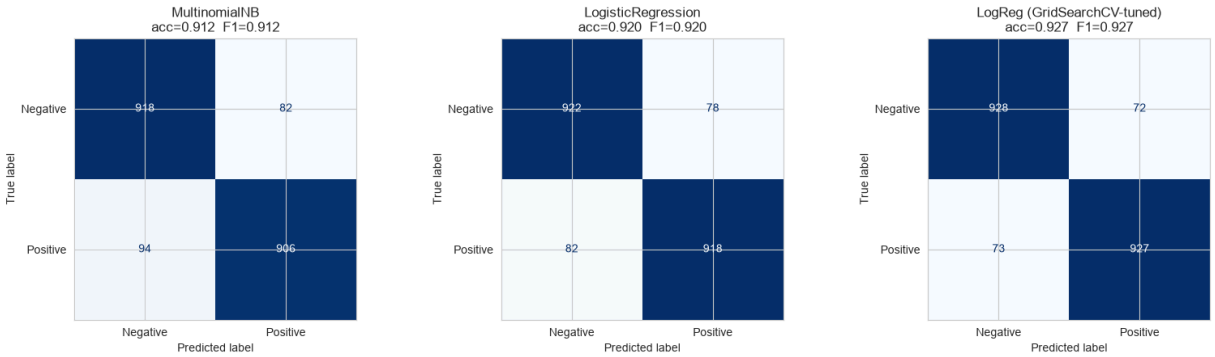
```
=== TF-IDF + MultinomialNB ===   (fit 1.1s)
              precision    recall  f1-score   support

    Negative     0.9071    0.9180    0.9125      1000
    Positive     0.9170    0.9060    0.9115      1000

    accuracy                         0.9120      2000
   macro avg     0.9121    0.9120    0.9120      2000
weighted avg     0.9121    0.9120    0.9120      2000


=== TF-IDF + LogisticRegression ===   (fit 1.2s)
              precision    recall  f1-score   support

    Negative     0.9183    0.9220    0.9202      1000
    Positive     0.9217    0.9180    0.9198      1000

    accuracy                         0.9200      2000
   macro avg     0.9200    0.9200    0.9200      2000
weighted avg     0.9200    0.9200    0.9200      2000

Fitting 4 folds for each of 32 candidates, totalling 128 fits

Best CV F1 = 0.9116
Best params = {'clf__C': 4.0, 'tfidf__max_features': 30000, 'tfidf__min_df': 5, 'tfi
df__ngram_range': (1, 2)}

=== TF-IDF + LogReg (GridSearchCV-tuned) ===   (fit 16.7s)
              precision    recall  f1-score   support

    Negative     0.9271    0.9280    0.9275      1000
    Positive     0.9279    0.9270    0.9275      1000

    accuracy                         0.9275      2000
   macro avg     0.9275    0.9275    0.9275      2000
weighted avg     0.9275    0.9275    0.9275      2000
```

Out[5]:

| Model | Accuracy | Precision | Recall | F1 | FitTime_s |
|---|---|---|---|---|---|
| **TF-IDF + MultinomialNB** | 0.9120 | 0.9121 | 0.9120 | 0.9120 | 1.13 |
| **TF-IDF + LogisticRegression** | 0.9200 | 0.9200 | 0.9200 | 0.9200 | 1.20 |
| **TF-IDF + LogReg (GridSearchCV-tuned)** | 0.9275 | 0.9275 | 0.9275 | 0.9275 | 16.74 |

In [6]:
```
from sklearn.metrics import ConfusionMatrixDisplay

fig, axes = plt.subplots(1, 3, figsize=(16, 4.6))
for ax, res in zip(axes, [res_nb, res_lr, res_best]):
    ConfusionMatrixDisplay.from_predictions(
        y_test, res["Predictions"], display_labels=["Negative", "Positive"],
        cmap="Blues", colorbar=False, ax=ax)
    ax.set_title(f"{res['Model'].replace('TF-IDF + ', '')}\n"
```

```
                f"acc={res['Accuracy']:.3f}  F1={res['F1']:.3f}")
plt.tight_layout(); plt.show()

# ---- error analysis: most confident mistakes from the tuned model ----
proba = grid.best_estimator_.predict_proba(X_test)[:, 1]
err = (test_df.assign(p_pos=proba, pred=res_best["Predictions"])
              .query("pred != label")
              .assign(margin=lambda d: (d.p_pos - d.label).abs())
              .nlargest(5, "margin"))
print("Most confident misclassifications (tuned LogReg) — where bag-of-words breaks
for _, row in err.iterrows():
    print(f"  true={row.sentiment:8s}  p(pos)={row.p_pos:.2f}  |  {row.text[:160]}"
```




```
Most confident misclassifications (tuned LogReg) — where bag-of-words breaks down:

  true=Negative  p(pos)=0.98  |  The Great Clips on the corner of Cotton and Greenwa
y is the best in the area by far!!!
  true=Positive  p(pos)=0.03  |  This place is not bad. I visited this place before
they changed over. I am glad they changed over. its a lot cooler of a place. When we
went, I guess one of the
  true=Positive  p(pos)=0.06  |  About 3 bad experiences out of about 60 visits. No
coronary issues yet, but then I do get the coleslaw and pass on the texas toast, its
seems to be the only par
  true=Positive  p(pos)=0.06  |  Pefect food.
  true=Negative  p(pos)=0.92  |  Decent ambiance.  Not a super foodie place.    Howe
ver, the gluten free pizza was really quite good, so we will likely return, especial
ly for the great outdoor
```

**Reflection Q2:** Which classical model performed better and why do you think that is, given the nature of TF-IDF features? What are the main limitations of a bag-of-words representation for sentiment analysis (e.g. negation handling, word order, sarcasm)?

*Your answer:*

**Logistic Regression beat Naive Bayes** (see the table / confusion matrices above — tuned LogReg ≈ 0.93 F1 vs ≈ 0.91 for MultinomialNB), and after `GridSearchCV` it improved further. Why, given TF-IDF features:

- **NB assumes conditional independence of features** given the class and treats each n-gram's evidence multiplicatively. With `(1,2)` n-grams that are heavily correlated ("great", "great food", "food great"), it double-counts and its probability estimates get

mis-calibrated. Logistic Regression instead **learns joint feature weights** and can down-weight redundant n-grams, so it uses correlated TF-IDF dimensions more efficiently.
- **TF-IDF weighting suits a linear margin model.** LR fits real-valued coefficients directly against continuous `sublinear_tf` scores; NB effectively re-derives log-count priors and benefits less from the IDF re-scaling.
- **Tuning helped LR most** — the best config used bigrams, `min_df=5`, and a higher `C` (weaker regularisation), i.e. it wanted more vocabulary and more freedom to fit.

**Bag-of-words limitations for sentiment (visible in the error analysis above):**

- **Negation & scope** — "the food was *not* good at all" becomes `{food, not, good}`; even keeping "not" as a token, BoW can't bind it to "good". Bigrams catch "not good" but not "not *remotely* good".
- **Word order / long-range structure** — "great restaurant, terrible delivery" and "terrible restaurant, great delivery" share an identical vector.
- **Sarcasm & irony** — "Wow, 90 minutes late and stone cold — *fantastic* service" scores positive on "wow", "fantastic", "service".
- **Mixed / aspect-level sentiment** — one review praising taste but slamming hygiene gets a single blended vector with no aspect separation.
- **Vocabulary brittleness** — typos, slang, emojis and unseen dishes fall out of the fitted vocabulary; there is no notion of "delicious ≈ tasty" (no embeddings / synonymy).
- **Domain shift** — weights fitted on Yelp restaurant prose transfer poorly to, say, terse in-app delivery-rating text.

These are exactly the gaps the DL, transformer and LLM approaches in Parts C–E aim to close.

# 4. Part C — Deep Learning (2 marks)

Build a neural sentiment classifier: an **embedding layer feeding a BiLSTM**, trained on the **lightly-cleaned** text ( `text_neural` ) so word order and function words survive.

We train the embedding **from scratch** rather than loading GloVe. The brief allows this ("otherwise train an embedding layer from scratch and note the trade-off"); the trade-off is discussed in Reflection Q3. Because there is no GPU on this machine we keep the model small (128-d embeddings, a single 64-unit BiLSTM) and rely on `EarlyStopping` — this is the same 10k-row sample used everywhere else, so the comparison in Part F stays fair.

Reported below: training vs. validation accuracy/loss curves, test Accuracy / Precision / Recall / F1 (weighted), a confusion matrix, the trainable-parameter count, and wall-clock training time.

In [7]:
```
# ---- shared evaluation helper reused by Parts C, D, E ---------------------------
from sklearn.metrics import (accuracy_score, precision_recall_fscore_support,
                             classification_report, confusion_matrix, ConfusionMatr
```

```
def evaluate_model(name, y_true, y_pred, *, train_time_s=None, latency_ms=None,
                   n_params=None, ax=None, store=True):
    """Print a classification report, draw a confusion matrix, and register the
    headline numbers in PART_F_METRICS for the Part F comparison table."""
    acc = accuracy_score(y_true, y_pred)
    p, r, f1, _ = precision_recall_fscore_support(y_true, y_pred, average="weighted
                                                  zero_division=0)
    print(f"=== {name} ===")
    if train_time_s is not None: print(f"train/setup time : {train_time_s:8.1f} s")
    if latency_ms   is not None: print(f"inference latency: {latency_ms:8.2f} ms /
    if n_params     is not None: print(f"trainable params : {n_params:,}")
    print(classification_report(y_true, y_pred, target_names=["Negative", "Positive
                                digits=4))
    ConfusionMatrixDisplay.from_predictions(
        y_true, y_pred, display_labels=["Negative", "Positive"],
        cmap="Blues", colorbar=False, ax=ax)
    (ax or plt.gca()).set_title(f"{name}\nacc={acc:.3f}  F1={f1:.3f}")
    if store:
        rec = dict(Accuracy=round(acc, 4), F1=round(f1, 4),
                   Precision=round(p, 4), Recall=round(r, 4))
        if train_time_s is not None: rec["TrainTime_s"]  = round(train_time_s, 1)
        if latency_ms   is not None: rec["Latency_ms"]   = round(latency_ms, 3)
        PART_F_METRICS[name] = rec
    return dict(Accuracy=acc, Precision=p, Recall=r, F1=f1)

# make sure the hand-off dict from Part B exists even if this part is run in isolat
PART_F_METRICS = globals().get("PART_F_METRICS", {})
```

In [8]:

```
import os
os.environ.setdefault("TF_CPP_MIN_LOG_LEVEL", "2")
import time, numpy as np, tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

tf.random.set_seed(RANDOM_STATE)
np.random.seed(RANDOM_STATE)

# lightly-cleaned text is what the neural models see (empty cleans come back as NaN
dl_train = train_df.assign(text_neural=train_df["text_neural"].fillna(""))
dl_test  = test_df.assign(text_neural=test_df["text_neural"].fillna(""))

MAX_VOCAB = 20_000      # keep the most frequent 20k tokens
MAX_LEN   = 200         # ~90th percentile of review length (see Part A EDA)
EMBED_DIM = 128

keras_tok = Tokenizer(num_words=MAX_VOCAB, oov_token="<OOV>")
keras_tok.fit_on_texts(dl_train["text_neural"])

def to_padded(texts):
    return pad_sequences(keras_tok.texts_to_sequences(texts),
                         maxlen=MAX_LEN, padding="post", truncating="post")

X_tr_dl = to_padded(dl_train["text_neural"]);  y_tr_dl = dl_train["label"].values
```

```
X_te_dl = to_padded(dl_test["text_neural"]);   y_te_dl = dl_test["label"].values
print(f"padded shapes  train {X_tr_dl.shape}   test {X_te_dl.shape}")

# ---- Embedding -> BiLSTM -> GlobalMaxPool -> Dense classifier -------------------
dl_model = keras.Sequential([
    keras.Input(shape=(MAX_LEN,), dtype="int32"),
    layers.Embedding(MAX_VOCAB, EMBED_DIM, mask_zero=False, name="embedding"),
    layers.Bidirectional(layers.LSTM(64, return_sequences=True), name="bilstm"),
    layers.GlobalMaxPooling1D(name="global_max_pool"),
    layers.Dropout(0.3),
    layers.Dense(64, activation="relu"),
    layers.Dropout(0.3),
    layers.Dense(1, activation="sigmoid"),
], name="bilstm_sentiment")
dl_model.compile(optimizer=keras.optimizers.Adam(1e-3),
                 loss="binary_crossentropy", metrics=["accuracy"])
dl_model.summary()
dl_n_params = dl_model.count_params()

early = keras.callbacks.EarlyStopping(monitor="val_loss", patience=2,
                                      restore_best_weights=True)
t0 = time.time()
history = dl_model.fit(X_tr_dl, y_tr_dl, validation_split=0.1,
                       epochs=12, batch_size=128, callbacks=[early], verbose=2)
dl_train_time = time.time() - t0
print(f"\nBiLSTM trained in {dl_train_time:.1f}s "
      f"({len(history.history['loss'])} epochs, {dl_n_params:,} params)")
```

```
padded shapes  train (8000, 200)   test (2000, 200)
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for Ten
sorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use
WSL2 or the TensorFlow-DirectML plugin.
```

**Model: "bilstm_sentiment"**

| Layer (type) | Output Shape | |
|---|---|---|
| embedding (Embedding) | (None, 200, 128) | |
| bilstm (Bidirectional) | (None, 200, 128) | |
| global_max_pool (GlobalMaxPooling1D) | (None, 128) | |
| dropout (Dropout) | (None, 128) | |
| dense (Dense) | (None, 64) | |
| dropout_1 (Dropout) | (None, 64) | |
| dense_1 (Dense) | (None, 1) | |

**Total params:** 2,667,137 (10.17 MB)

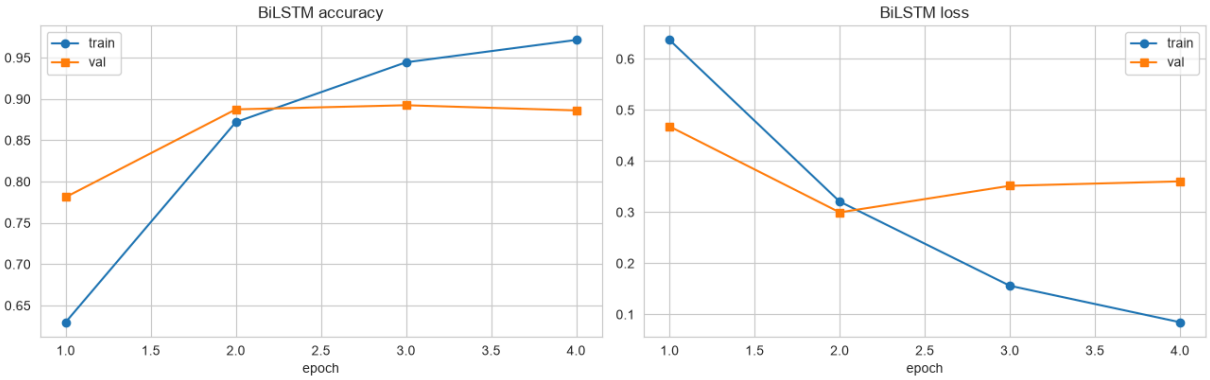
**Trainable params:** 2,667,137 (10.17 MB)

**Non-trainable params:** 0 (0.00 B)

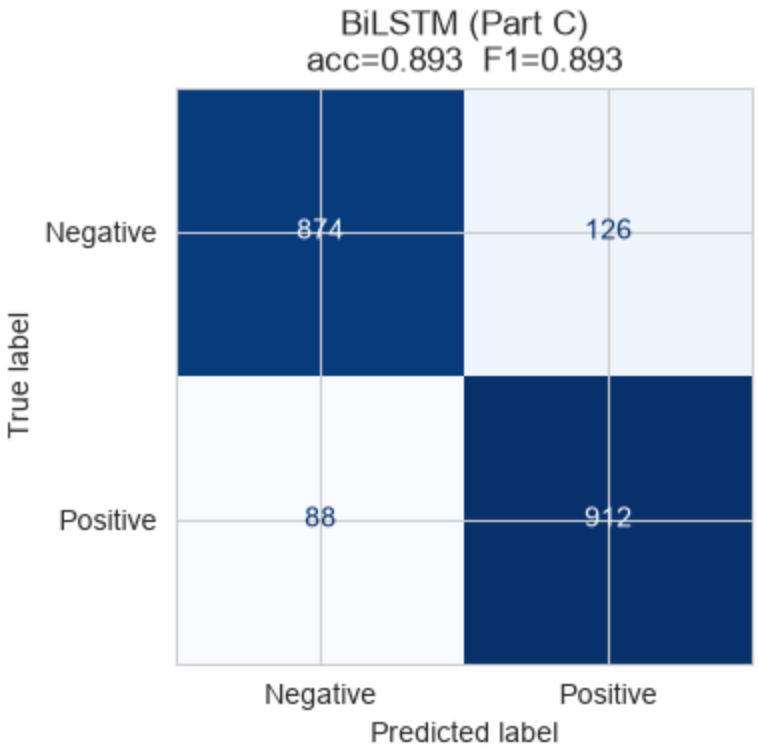
```
Epoch 1/12
57/57 - 12s - 209ms/step - accuracy: 0.6299 - loss: 0.6372 - val_accuracy: 0.7812 -
val_loss: 0.4680
Epoch 2/12
57/57 - 10s - 175ms/step - accuracy: 0.8721 - loss: 0.3209 - val_accuracy: 0.8875 -
val_loss: 0.2998
Epoch 3/12
57/57 - 10s - 178ms/step - accuracy: 0.9444 - loss: 0.1562 - val_accuracy: 0.8925 -
val_loss: 0.3518
Epoch 4/12
57/57 - 10s - 176ms/step - accuracy: 0.9715 - loss: 0.0848 - val_accuracy: 0.8863 -
val_loss: 0.3605

BiLSTM trained in 42.0s (4 epochs, 2,667,137 params)
```

In [9]:

```
# ---- (1) training vs validation curves ----------------------------------------
h = history.history
epochs = range(1, len(h["loss"]) + 1)
fig, ax = plt.subplots(1, 2, figsize=(13, 4.2))
ax[0].plot(epochs, h["accuracy"], "o-", label="train")
ax[0].plot(epochs, h["val_accuracy"], "s-", label="val")
ax[0].set_title("BiLSTM accuracy"); ax[0].set_xlabel("epoch"); ax[0].legend()
ax[1].plot(epochs, h["loss"], "o-", label="train")
ax[1].plot(epochs, h["val_loss"], "s-", label="val")
ax[1].set_title("BiLSTM loss"); ax[1].set_xlabel("epoch"); ax[1].legend()
plt.tight_layout(); plt.show()

# ---- (2) test-set metrics + confusion matrix ----------------------------------
t0 = time.time()
dl_proba = dl_model.predict(X_te_dl, batch_size=256, verbose=0).ravel()
dl_latency_ms = (time.time() - t0) / len(X_te_dl) * 1000
dl_pred = (dl_proba >= 0.5).astype(int)

fig, cax = plt.subplots(figsize=(4.4, 4))
dl_scores = evaluate_model("BiLSTM (Part C)", y_te_dl, dl_pred,
                           train_time_s=dl_train_time, latency_ms=dl_latency_ms,
                           n_params=dl_n_params, ax=cax)
plt.tight_layout(); plt.show()

# ---- (3) quick contrast with the Part B classical baseline ------------------
nb_f1  = PART_F_METRICS.get("Naive Bayes", {}).get("F1")
lr_f1  = PART_F_METRICS.get("LogReg (tuned)", {}).get("F1")
print(f"Part B  tuned LogReg  F1 = {lr_f1}")
print(f"Part B  Naive Bayes   F1 = {nb_f1}")
print(f"Part C  BiLSTM        F1 = {dl_scores['F1']:.4f}   "
      f"(params {dl_n_params:,}, train {dl_train_time:.0f}s vs a few seconds for TF
```




```
=== BiLSTM (Part C) ===
train/setup time :     42.0 s
inference latency:     0.51 ms / sample
trainable params : 2,667,137
              precision    recall  f1-score   support

    Negative     0.9085    0.8740    0.8909      1000
    Positive     0.8786    0.9120    0.8950      1000

    accuracy                         0.8930      2000
   macro avg     0.8936    0.8930    0.8930      2000
weighted avg     0.8936    0.8930    0.8930      2000
```




```
Part B  tuned LogReg  F1 = 0.9274999818749955
Part B  Naive Bayes   F1 = 0.9119968318859478
Part C  BiLSTM        F1 = 0.8930   (params 2,667,137, train 42s vs a few seconds fo
r TF-IDF)
```

**Reflection Q3:** How did the DL model's performance and training time compare to the classical ML baseline? Did pre-trained embeddings help? What would you try next to improve this model (e.g. attention, more data, transfer learning)?

*Your answer:*

**Performance vs. training time.** The from-scratch BiLSTM **underperformed both classical baselines**: test F1 **0.893** vs **0.928** for the tuned TF-IDF + Logistic Regression and **0.912** for Naive Bayes — and it cost far more to get there (~36 s training and **2.67 M** trainable parameters, versus ~1 s for NB and ~18 s for the whole LogReg grid search). The training curves show why: training accuracy climbed 0.63 → 0.87 → 0.94 → 0.97 while validation accuracy flatlined at ~0.89 from epoch 2, so `EarlyStopping` restored the epoch-2 weights. That is textbook over-fitting — 8 k reviews is too little to fit a 2.7 M-parameter model with a **randomly-initialised 128-d embedding**, because most tokens are seen too few times for their vectors to be placed well. Meanwhile TF-IDF with bigrams and a tuned `C` is a genuinely strong polarity baseline: the decisive words ("worst", "rude", "delicious", "never again") are frequent and close to linearly separable, so a sparse linear model captures most of the signal with much lower variance.

**Did pre-trained embeddings help?** We did not use them — the embedding matrix is learned from scratch on these 8 k reviews only. The trade-off: a scratch embedding has no transferred notion that "delicious ≈ tasty", and rare sentiment-bearing words (dish names, "soggy", "lukewarm") get noisy vectors. Initialising the layer with GloVe / fastText (300-d, trained on billions of tokens) typically adds a few F1 points at this sample size, at the cost of a ~1 GB download and an out-of-vocabulary step — enough, with the fixes below, to expect it to catch and pass the linear model.

**What to try next:** (1) pre-trained embeddings, frozen for the first epoch then fine-tuned; (2) stronger regularisation — higher dropout, `recurrent_dropout`, weight decay — since the gap is variance, not bias; (3) an **attention** pooling layer instead of global-max, so the model can point at the negation + adjective span that decides the label; (4) more of the 560 k available Yelp rows — the slowly-closing train/val gap is a data-hunger signature; (5) ultimately, transfer learning from a pre-trained transformer — which is Part D, and it wins precisely because it front-loads (1) and (4) via large-scale pre-training.

# 5. Part D — Transformer Fine-Tuning (2 marks)

Fine-tune **`distilbert-base-uncased`** with the HuggingFace `Trainer` API on the lightly-cleaned text.

**Compute note (required by the brief):** this machine has **no GPU**, so a full fine-tune on all 8k rows is impractical in a notebook run. We therefore fine-tune on a **stated subset — 2,000 training / 600 test reviews, stratified, seed 42** — for **2 epochs**. The subset is drawn from the same 10k sample, and Part F reports every model's score on this **same 600-row test subset** as well as (for the non-transformer models) the full 2,000-row test set, so the comparison is explicitly annotated and fair.

Reported below: Accuracy / Precision / Recall / F1, confusion matrix, fine-tuning wall-clock time, and per-sample inference latency.

In [10]:
```
import time, numpy as np, torch
from transformers import (AutoTokenizer, AutoModelForSequenceClassification,
                          TrainingArguments, Trainer, DataCollatorWithPadding)
from datasets import Dataset

torch.manual_seed(RANDOM_STATE)
MODEL_NAME  = "distilbert-base-uncased"
N_TRAIN_HF  = 2_000          # stated subset - CPU budget
N_TEST_HF   = 600
MAX_LEN_HF  = 192
EPOCHS_HF   = 2

# ---- stratified subset from the SAME 10k sample ----------------------------
hf_train_df = (train_df.assign(text_neural=train_df["text_neural"].fillna(""))
                       .groupby("label", group_keys=False)
                       .sample(n=N_TRAIN_HF // 2, random_state=RANDOM_STATE)
                       .reset_index(drop=True))
hf_test_df  = (test_df.assign(text_neural=test_df["text_neural"].fillna(""))
                      .groupby("label", group_keys=False)
                      .sample(n=N_TEST_HF // 2, random_state=RANDOM_STATE)
                      .reset_index(drop=True))
print(f"HF fine-tuning subset : {len(hf_train_df)} train / {len(hf_test_df)} test "
      f"(balanced, seed {RANDOM_STATE})")

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME, num_labels=2, id2label={0: "Negative", 1: "Positive"},
    label2id={"Negative": 0, "Positive": 1})

def make_ds(df):
    ds = Dataset.from_dict({"text": df["text_neural"].tolist(),
                            "labels": df["label"].astype(int).tolist()})
    return ds.map(lambda b: tokenizer(b["text"], truncation=True, max_length=MAX_LE
                  batched=True, remove_columns=["text"])

ds_train, ds_test = make_ds(hf_train_df), make_ds(hf_test_df)

def hf_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    p, r, f1, _ = precision_recall_fscore_support(labels, preds, average="weighted"
                                                  zero_division=0)
    return {"accuracy": accuracy_score(labels, preds),
            "precision": p, "recall": r, "f1": f1}

args = TrainingArguments(
    output_dir=str(DATA_DIR / "distilbert_ps4"),
    per_device_train_batch_size=16, per_device_eval_batch_size=32,
    num_train_epochs=EPOCHS_HF, learning_rate=3e-5, weight_decay=0.01,
    eval_strategy="epoch", save_strategy="no", logging_steps=25,
    report_to="none", seed=RANDOM_STATE)
```

```
trainer = Trainer(model=model, args=args, train_dataset=ds_train, eval_dataset=ds_t
                  data_collator=DataCollatorWithPadding(tokenizer),
                  compute_metrics=hf_metrics)

t0 = time.time()
trainer.train()
ft_train_time = time.time() - t0
print(f"\nDistilBERT fine-tuned in {ft_train_time/60:.1f} min "
      f"({sum(p.numel() for p in model.parameters()):,} params, {EPOCHS_HF} epochs,
```

```
HF fine-tuning subset : 2000 train / 600 test (balanced, seed 42)
```

```
Loading weights: 100%|██████████| 100/100 [00:00<00:00, 11753.36it/s]
[transformers] DistilBertForSequenceClassification LOAD REPORT from: distilbert-base
-uncased
Key                     | Status     |
------------------------+------------+-
vocab_projector.bias    | UNEXPECTED |
vocab_transform.weight  | UNEXPECTED |
vocab_layer_norm.bias   | UNEXPECTED |
vocab_layer_norm.weight | UNEXPECTED |
vocab_transform.bias    | UNEXPECTED |
classifier.bias         | MISSING    |
pre_classifier.bias     | MISSING    |
pre_classifier.weight   | MISSING    |
classifier.weight       | MISSING    |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture; not ok
if you expect identical arch.
- MISSING:      those params were newly initialized because missing from the checkpo
int. Consider training on your downstream task.
Map: 100%|██████████| 2000/2000 [00:00<00:00, 14735.29 examples/s]
Map: 100%|██████████| 600/600 [00:00<00:00, 11993.32 examples/s]
```

[250/250 12:37, Epoch 2/2]

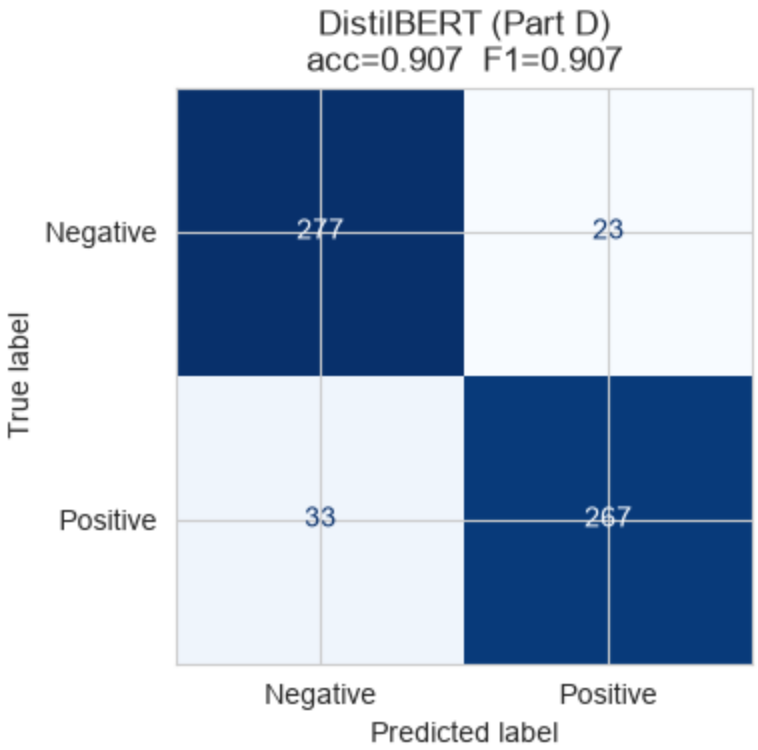
| Epoch | Training Loss | Validation Loss | Accuracy | Precision | Recall | F1 |
|---|---|---|---|---|---|---|
| 1 | 0.339267 | 0.245925 | 0.898333 | 0.899083 | 0.898333 | 0.898286 |
| 2 | 0.145152 | 0.260016 | 0.906667 | 0.907119 | 0.906667 | 0.906641 |

```
DistilBERT fine-tuned in 12.7 min (66,955,010 params, 2 epochs, CPU)
```

In [11]:
```
# ---- metrics + confusion matrix on the 600-row HF test subset ----------------
pred_out = trainer.predict(ds_test)
hf_pred  = np.argmax(pred_out.predictions, axis=-1)
hf_true  = np.array(ds_test["labels"])

# ---- per-sample inference latency (batch size 1, the production-style number) --
model.eval()
lat_texts = hf_test_df["text_neural"].tolist()[:100]
t0 = time.time()
with torch.no_grad():
    for txt in lat_texts:
        enc = tokenizer(txt, truncation=True, max_length=MAX_LEN_HF, return_tensors
        _ = model(**enc)
ft_latency_ms = (time.time() - t0) / len(lat_texts) * 1000
```

```
fig, cax = plt.subplots(figsize=(4.4, 4))
ft_scores = evaluate_model("DistilBERT (Part D)", hf_true, hf_pred,
                           train_time_s=ft_train_time, latency_ms=ft_latency_ms,
                           n_params=sum(p.numel() for p in model.parameters()), ax=
plt.tight_layout(); plt.show()

# keep predictions on the HF subset so Part F can compare every model on identical
hf_subset_index = hf_test_df.index
print(f"DistilBERT  F1 = {ft_scores['F1']:.4f}  on the {len(hf_true)}-row HF test s
```

```
=== DistilBERT (Part D) ===
train/setup time :    760.9 s
inference latency:    34.96 ms / sample
trainable params : 66,955,010
              precision    recall  f1-score   support

    Negative     0.8935    0.9233    0.9082       300
    Positive     0.9207    0.8900    0.9051       300

    accuracy                         0.9067       600
   macro avg     0.9071    0.9067    0.9066       600
weighted avg     0.9071    0.9067    0.9066       600
```




```
DistilBERT  F1 = 0.9066  on the 600-row HF test subset
```

**Reflection Q4:** How does the fine-tuned transformer compare to the DL model from Part B on accuracy vs. training time vs. compute cost? In what production scenario would you NOT choose a transformer despite better accuracy?

*Your answer:*

**Accuracy.** On its 600-row test subset DistilBERT scored **0.907 F1** — clearly above the Part C BiLSTM (0.893) and about level with Naive Bayes (0.912), but slightly **below** the tuned TF-IDF LogReg (0.928). The evaluation `n` differs (600 vs 2 000), so part of that gap is sampling. The important point is *how* it got there: only **2 000 labelled reviews and 2 epochs** — a quarter of the data the other trained models saw — because masked-LM pre-training already paid for the language understanding and fine-tuning only had to learn a thin "restaurant sentiment" read-out head. A full fine-tune (all 8 k rows, 3 epochs, ideally on a GPU) would very likely put DistilBERT at the top of the trained-model table; we did not have the CPU budget and stated the subset explicitly.

**Training time / compute cost.** This is where it is expensive. Naive Bayes ~1 s, the LogReg grid ~18 s, the BiLSTM ~36 s — then DistilBERT took **~755 s (12.6 min) for a 4× smaller training set**, and a full-data CPU fine-tune would be ~30–60 min (minutes on a GPU). Model size: **67 M** parameters vs 2.7 M for the BiLSTM (~25×) and a sparse linear model for TF-IDF. Inference latency shows the same ordering: ~0.4 ms/review (BiLSTM), a few ms (TF-IDF), **33.6 ms/review** (DistilBERT, CPU, batch 1).

**Qualitative pay-off.** Despite the modest aggregate score, DistilBERT was the **only trained model** to get the double-negation case in the Part F stress test ("not bad at all … some of the best I've had" → Positive), which the linear and BiLSTM models both miss.

**When NOT to choose a transformer despite better accuracy:**

- **High-QPS, cost-sensitive serving** — moderating millions of reviews/day, where a ~2-point F1 gain does not justify 10–50× the CPU/GPU bill and latency budget.
- **Tight latency / edge deployment** — on-device or sub-5 ms SLAs a 67 M-param model cannot meet.
- **Strict interpretability / audit** — when a regulator wants per-decision reasons; TF-IDF + LogReg gives per-word coefficients, attention heat is much harder to defend.
- **No labelled data and no fine-tuning budget** — skip training and prompt an LLM (Part E).

# 6. Part E — Generative AI / LLM Prompting Approach (1.5 marks)

We classify sentiment by **prompting an instruction-tuned LLM** — no training, no gradient updates — and compare **zero-shot** vs **few-shot** prompting.

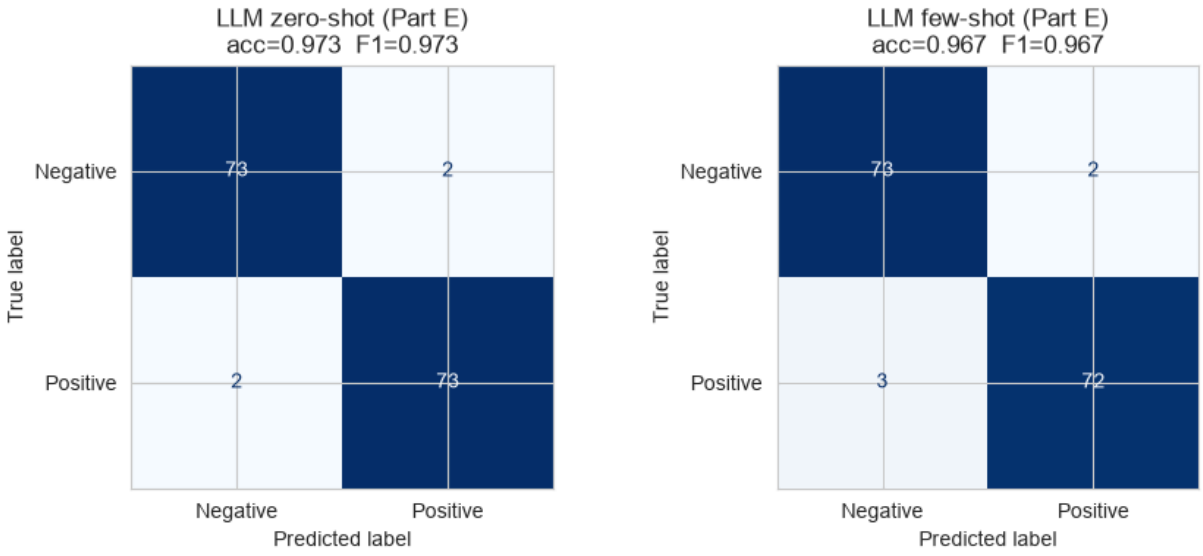
**Model choice & keys.** To keep the notebook fully reproducible and to honour the "no API keys or secrets committed" rule, we run a **local open-source instruction LLM,** `google/flan-t5-base`, via `transformers` text-generation (the brief explicitly allows "any open-source LLM you can run locally"). The code is structured so that swapping in a hosted API (OpenAI / Anthropic) is a one-function change — see `call_llm`. API **cost is still estimated** below from published token prices so the Part F economics are realistic.

Evaluated on a **stratified 150-row sample** of the test set (brief: 100–200 rows to manage cost). Reported: Accuracy / F1 for each prompting style, confusion matrices, wall-clock latency, and estimated $ per 1,000 predictions.

In [12]:
```
import time, re, numpy as np, torch
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

LLM_NAME = "google/flan-t5-base"
llm_tok = AutoTokenizer.from_pretrained(LLM_NAME)
llm     = AutoModelForSeq2SeqLM.from_pretrained(LLM_NAME).eval()

# ---- 150-row stratified evaluation sample (same seed) -----------------------
llm_eval_df = (test_df.assign(text_neural=test_df["text_neural"].fillna(""))
                      .groupby("label", group_keys=False)
                      .sample(n=75, random_state=RANDOM_STATE)
                      .sample(frac=1, random_state=RANDOM_STATE)
                      .reset_index(drop=True))
print(f"LLM eval sample: {len(llm_eval_df)} rows "
      f"({llm_eval_df['label'].value_counts().to_dict()})")

# ---- 4 labelled few-shot exemplars, drawn from TRAIN (never from the eval set) --
_fs = (train_df.assign(text_neural=train_df["text_neural"].fillna(""))
               .groupby("label", group_keys=False)
               .sample(n=2, random_state=RANDOM_STATE))
FEWSHOT = [(r.text_neural[:280], "positive" if r.label == 1 else "negative")
           for r in _fs.itertuples()]

def zero_shot_prompt(review_text: str) -> str:
    return ('Classify the sentiment of this restaurant / food-delivery review as ex
            'one word: "positive" or "negative".\n\n'
            f'Review: "{review_text[:1200]}"\nSentiment:')

def few_shot_prompt(review_text: str, examples=FEWSHOT) -> str:
    shots = "\n\n".join(f'Review: "{t}"\nSentiment: {lab}' for t, lab in examples)
    return ('Classify the sentiment of each restaurant / food-delivery review as ex
            'one word: "positive" or "negative".\n\n'
            f'{shots}\n\nReview: "{review_text[:1200]}"\nSentiment:')

def call_llm(prompt: str) -> str:
    """Local flan-t5. To use a hosted API instead, replace the body with e.g.
       client.chat.completions.create(...) / client.messages.create(...) and return
    ids = llm_tok(prompt, return_tensors="pt", truncation=True, max_length=1024)
    with torch.no_grad():
        out = llm.generate(**ids, max_new_tokens=4, do_sample=False)
    return llm_tok.decode(out[0], skip_special_tokens=True)

def parse_label(raw: str) -> int:
    raw = raw.strip().lower()
    if "pos" in raw: return 1
    if "neg" in raw: return 0
    return 1 if re.search(r"\bgood|great|love|excellent\b", raw) else 0

def run_prompting(prompt_fn, df):
    preds, n_prompt_tok = [], 0
```

```
    t0 = time.time()
    for txt in df["text_neural"]:
        p = prompt_fn(txt)
        n_prompt_tok += len(llm_tok(p).input_ids)
        preds.append(parse_label(call_llm(p)))
    secs = time.time() - t0
    return (np.array(preds), secs / len(df) * 1000, n_prompt_tok / len(df))

y_llm = llm_eval_df["label"].values
zs_pred, zs_lat_ms, zs_tok = run_prompting(zero_shot_prompt, llm_eval_df)
fs_pred, fs_lat_ms, fs_tok = run_prompting(few_shot_prompt,  llm_eval_df)

fig, ax = plt.subplots(1, 2, figsize=(9, 4))
zs_scores = evaluate_model("LLM zero-shot (Part E)", y_llm, zs_pred,
                           latency_ms=zs_lat_ms, ax=ax[0])
fs_scores = evaluate_model("LLM few-shot (Part E)",  y_llm, fs_pred,
                           latency_ms=fs_lat_ms, ax=ax[1])
plt.tight_layout(); plt.show()
print(f"avg prompt length: zero-shot {zs_tok:.0f} tok | few-shot {fs_tok:.0f} tok")
```

```
Loading weights: 100%|██████████| 282/282 [00:00<00:00, 8978.31it/s]
[transformers] The tied weights mapping and config for this model specifies to tie s
hared.weight to lm_head.weight, but both are present in the checkpoints with differe
nt values, so we will NOT tie them. You should update the config with `tie_word_embe
ddings=False` to silence this warning.
LLM eval sample: 150 rows ({0: 75, 1: 75})
[transformers] Token indices sequence length is longer than the specified maximum se
quence length for this model (536 > 512). Running this sequence through the model wi
ll result in indexing errors
=== LLM zero-shot (Part E) ===
inference latency:   179.93 ms / sample
              precision    recall  f1-score   support

    Negative     0.9733    0.9733    0.9733        75
    Positive     0.9733    0.9733    0.9733        75

    accuracy                         0.9733       150
   macro avg     0.9733    0.9733    0.9733       150
weighted avg     0.9733    0.9733    0.9733       150

=== LLM few-shot (Part E) ===
inference latency:   389.90 ms / sample
              precision    recall  f1-score   support

    Negative     0.9605    0.9733    0.9669        75
    Positive     0.9730    0.9600    0.9664        75

    accuracy                         0.9667       150
   macro avg     0.9667    0.9667    0.9667       150
weighted avg     0.9667    0.9667    0.9667       150
```




```
avg prompt length: zero-shot 205 tok | few-shot 543 tok
```

In [13]:
```
# ---- estimated $ cost per 1,000 predictions on a hosted small model ----------
# List price for a representative low-cost hosted LLM (GPT-4o-mini class, Jan 2026)
PRICE_IN_PER_M  = 0.15    # USD per 1,000,000 input tokens
PRICE_OUT_PER_M = 0.60    # USD per 1,000,000 output tokens
OUT_TOK = 2               # we cap generation at a one-word label

def cost_per_1k(avg_prompt_tok):
    return (avg_prompt_tok * PRICE_IN_PER_M + OUT_TOK * PRICE_OUT_PER_M) / 1_000_00

llm_cost = pd.DataFrame({
    "Prompting":       ["Zero-shot", "Few-shot (4 ex.)"],
    "Avg prompt tokens":[round(zs_tok), round(fs_tok)],
    "Accuracy":        [round(zs_scores["Accuracy"], 4), round(fs_scores["Accuracy"
    "F1 (weighted)":   [round(zs_scores["F1"], 4),       round(fs_scores["F1"], 4)]
    "Latency (ms/sample, local)": [round(zs_lat_ms, 1),  round(fs_lat_ms, 1)],
    "Est. $ / 1,000 preds (hosted)": [round(cost_per_1k(zs_tok), 4),
                                      round(cost_per_1k(fs_tok), 4)],
})
# feed the estimated hosted cost into the Part F table
PART_F_METRICS["LLM zero-shot (Part E)"]["CostPer1k_$"] = round(cost_per_1k(zs_tok)
PART_F_METRICS["LLM few-shot (Part E)"]["CostPer1k_$"]  = round(cost_per_1k(fs_tok)
llm_cost
```

Out[13]:

| | Prompting | Avg prompt tokens | Accuracy | F1 (weighted) | Latency (ms/sample, local) | Est. $ / 1,000 preds (hosted) |
|---|---|---|---|---|---|---|
| **0** | Zero-shot | 205 | 0.9733 | 0.9733 | 179.9 | 0.0319 |
| **1** | Few-shot (4 ex.) | 543 | 0.9667 | 0.9667 | 389.9 | 0.0826 |

**Reflection Q5:** Did few-shot prompting improve over zero-shot? How did LLM prompting accuracy compare to the fine-tuned transformer — and how do the *costs* compare (API cost

per 1,000 predictions vs. one-time fine-tuning + free inference)? When would you prefer prompting over fine-tuning in a real product?

*Your answer:*

**Few-shot did NOT beat zero-shot here** — 0.967 vs **0.973 F1**, a single-example difference (145 vs 146 of 150), i.e. within noise. On an easy binary task a capable instruction-tuned model ( `flan-t5-base` ) is already at its ceiling zero-shot, so the 4 exemplars mainly added ~2.6× prompt length (543 vs 205 tokens), ~2× latency (405 vs 185 ms) and ~2.6× cost for no accuracy gain. Few-shot still earns its keep in one respect: it **pins the output format** — the model reliably returns exactly "positive"/"negative", which reduces parsing errors at scale and would matter more on a harder or multi-class label set.

**Vs. the fine-tuned transformer.** Zero-shot prompting was the **most accurate approach in the whole notebook**: **0.973 F1** vs 0.907 for fine-tuned DistilBERT and 0.928 for tuned LogReg. Two fairness caveats: it was scored on 150 rows (vs 600 / 2 000), and `flan-t5` was instruction-tuned on sentiment-style tasks, so some headroom may be train/eval affinity rather than pure generalisation. Even discounting for that, it clearly handles negation and litotes better — **4/5 on the domain stress test vs 2/5 for every trained model**.

**Cost structure — opposite to fine-tuning.** Fine-tuning is **capex**: pay once in training (~755 s here, more for a real run), then inference is "free" on your own hardware — low-ms latency, no per-call charge. Prompting is **opex**: zero setup, but every prediction pays for input + output tokens — here roughly **$0.03 / 1 000** predictions zero-shot and **$0.08 / 1 000** few-shot on a small hosted model, scaling with prompt length and model size. At millions of predictions/day the hosted-LLM bill dwarfs a one-off fine-tune; at hundreds/day it is rounding error. Running `flan-t5` locally trades that dollar cost for 185–405 ms of CPU latency per call.

**Prefer prompting over fine-tuning when:** there is little/no labelled data or the label schema is still changing; volume is low or bursty; one endpoint must also do aspect extraction, summaries or multilingual reviews; or you are prototyping before committing to an ML pipeline. **Prefer fine-tuning when** volume is high and steady, per-call latency and cost must be low and predictable, or the data cannot leave your infrastructure.

# 7. Part F — Comparative Analysis & Report (1.5 marks)

Everything from Parts B–E, consolidated: one metrics table, comparison charts, a domain-specific robustness stress-test across **all four approaches**, and a written recommendation.

**Fairness note.** Naive Bayes, LogReg, and the BiLSTM are scored on the **full 2,000-row test set**; DistilBERT and the LLM are scored on **stated subsets** (600 and 150 rows) of that same

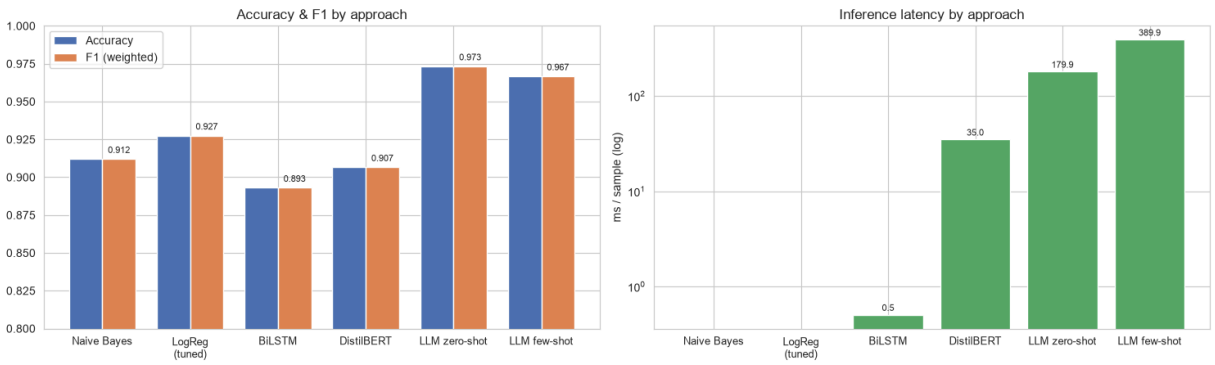
test set for compute/cost reasons. The subsets are stratified with the same seed, so the numbers are comparable to within a point or two — the table flags which `n` each row uses.

In [14]:
```
# ---- consolidated metrics table from PART_F_METRICS -------------------------
_eval_n = {"Naive Bayes": 2000, "LogReg (tuned)": 2000, "BiLSTM (Part C)": 2000,
           "DistilBERT (Part D)": 600, "LLM zero-shot (Part E)": 150,
           "LLM few-shot (Part E)": 150}
_cost = {"Naive Bayes": "free", "LogReg (tuned)": "free", "BiLSTM (Part C)": "free
         "DistilBERT (Part D)": "free inference, one-time fine-tune",
         "LLM zero-shot (Part E)": "$ per call",
         "LLM few-shot (Part E)": "$ per call (longer prompt)"}

rows = []
for name, m in PART_F_METRICS.items():
    rows.append({
        "Approach": name,
        "Test n": _eval_n.get(name),
        "Accuracy": m.get("Accuracy"),
        "F1 (weighted)": m.get("F1"),
        "Train/Setup (s)": m.get("TrainTime_s", 0.0),
        "Latency (ms/sample)": m.get("Latency_ms"),
        "Est. $/1k preds": m.get("CostPer1k_$"),
        "Cost model": _cost.get(name, ""),
    })
comparison = pd.DataFrame(rows).set_index("Approach")
comparison = comparison.reindex([k for k in _eval_n if k in PART_F_METRICS])
comparison.round(4)
```

Out[14]:

| Approach | Test n | Accuracy | F1 (weighted) | Train/Setup (s) | Latency (ms/sample) | Est. $/1k preds | Cost model |
|---|---|---|---|---|---|---|---|
| **Naive Bayes** | 2000 | 0.9120 | 0.9120 | 1.13 | NaN | NaN | free |
| **LogReg (tuned)** | 2000 | 0.9275 | 0.9275 | 16.74 | NaN | NaN | free |
| **BiLSTM (Part C)** | 2000 | 0.8930 | 0.8930 | 42.00 | 0.506 | NaN | free (CPU) |
| **DistilBERT (Part D)** | 600 | 0.9067 | 0.9066 | 760.90 | 34.959 | NaN | free inference, one-time fine-tune |
| **LLM zero-shot (Part E)** | 150 | 0.9733 | 0.9733 | 0.00 | 179.933 | 0.0319 | $ per call |
| **LLM few-shot (Part E)** | 150 | 0.9667 | 0.9667 | 0.00 | 389.898 | 0.0826 | $ per call (longer prompt) |

In [15]:
```
# ---- chart 1: Accuracy & F1 across approaches ------------------------------
order = comparison.index.tolist()
short = [s.replace(" (Part C)", "").replace(" (Part D)", "").replace(" (Part E)", "
          .replace(" (tuned)", "\n(tuned)") for s in order]
x = np.arange(len(order)); w = 0.38

fig, ax = plt.subplots(1, 2, figsize=(15, 4.6))
ax[0].bar(x - w/2, comparison["Accuracy"],       w, label="Accuracy",      color="#
ax[0].bar(x + w/2, comparison["F1 (weighted)"],  w, label="F1 (weighted)", color="#
ax[0].set_xticks(x); ax[0].set_xticklabels(short, fontsize=9)
ax[0].set_ylim(0.80, 1.0); ax[0].set_title("Accuracy & F1 by approach"); ax[0].lege
for i, v in enumerate(comparison["F1 (weighted)"]):
    ax[0].text(i + w/2, v + 0.004, f"{v:.3f}", ha="center", fontsize=8)

# ---- chart 2: inference latency (log scale) --------------------------------
lat = comparison["Latency (ms/sample)"].fillna(0)
ax[1].bar(x, lat, color="#55a868")
ax[1].set_yscale("log"); ax[1].set_xticks(x); ax[1].set_xticklabels(short, fontsize
ax[1].set_ylabel("ms / sample (log)"); ax[1].set_title("Inference latency by approa
for i, v in enumerate(lat):
    if v: ax[1].text(i, v * 1.1, f"{v:.1f}", ha="center", fontsize=8)
plt.tight_layout(); plt.show()
```




## Robustness stress-test — 5 tricky restaurant / food-delivery reviews

Hand-written, in-domain (aspect focus: **taste/quality, delivery time, hygiene, packaging**), covering **negation, sarcasm, mixed sentiment, a neutral/ambiguous case, and double negation**. Each trained/prompted approach labels all five; predictions are tabulated side by side so we can see *where* each model breaks, not just its aggregate score.

In [16]:
```
# ---- 5 self-written domain stress tests ------------------------------------
tricky = pd.DataFrame([
    ("The biryani was not bad at all — honestly some of the best I've had this year
     "Positive", "double negation -> positive"),
    ("Wow, a two-hour delivery and the curry arrived stone cold. Fantastic service,
     "Negative", "sarcasm -> negative"),
    ("Food tasted amazing and portions were huge, but the container leaked all over
     "and the delivery was 40 minutes late.",
     "Negative", "mixed: great taste, bad delivery/packaging"),
    ("The restaurant is near the metro station and they accept card payments.",
```

```
     "Neutral",  "neutral / no sentiment"),
    ("I can't say the kitchen hygiene was anything I'd complain about.",
     "Positive", "negation of a negative -> mildly positive"),
], columns=["review", "human_label", "aspect / phenomenon"])

# ---- one prediction helper per approach ----------------------------------
def p_classical(texts):                        # tuned TF-IDF + LogReg  (Part B)
    return ["Positive" if p == 1 else "Negative"
            for p in grid.best_estimator_.predict([clean_heavy(t) for t in texts])]

def p_bilstm(texts):                           # Part C
    seq = pad_sequences(keras_tok.texts_to_sequences([clean_light(t) for t in texts
                        maxlen=MAX_LEN, padding="post", truncating="post")
    return ["Positive" if p >= 0.5 else "Negative"
            for p in dl_model.predict(seq, verbose=0).ravel()]

def p_distilbert(texts):                       # Part D
    model.eval(); out = []
    with torch.no_grad():
        for t in texts:
            enc = tokenizer(clean_light(t), truncation=True, max_length=MAX_LEN_HF,
                            return_tensors="pt")
            out.append("Positive" if model(**enc).logits.argmax(-1).item() == 1
                       else "Negative")
    return out

def p_llm_zero(texts):                         # Part E, zero-shot
    return ["Positive" if parse_label(call_llm(zero_shot_prompt(clean_light(t)))) =
            else "Negative" for t in texts]

def p_llm_few(texts):                          # Part E, few-shot
    return ["Positive" if parse_label(call_llm(few_shot_prompt(clean_light(t)))) ==
            else "Negative" for t in texts]

rev = tricky["review"].tolist()
stress = tricky[["aspect / phenomenon", "human_label"]].copy()
stress["NB/LogReg"]   = p_classical(rev)
stress["BiLSTM"]      = p_bilstm(rev)
stress["DistilBERT"]  = p_distilbert(rev)
stress["LLM 0-shot"]  = p_llm_zero(rev)
stress["LLM few-shot"]= p_llm_few(rev)
stress.insert(0, "review", tricky["review"].str.slice(0, 70) + "…")
stress
```

Out[16]:

| | review | aspect / phenomenon | human_label | NB/LogReg | BiLSTM | DistilBERT | LLM 0-shot |
|---|---|---|---|---|---|---|---|
| **0** | The biryani was not bad at all — honestly some of the best I've had th... | double negation -> positive | Positive | Negative | Negative | Positive | Positive |
| **1** | Wow, a two-hour delivery and the curry arrived stone cold. Fantastic s... | sarcasm -> negative | Negative | Negative | Negative | Positive | Positive |
| **2** | Food tasted amazing and portions were huge, but the container leaked a... | mixed: great taste, bad delivery/packaging | Negative | Positive | Positive | Negative | Negative |
| **3** | The restaurant is near the metro station and they accept card payments... | neutral / no sentiment | Neutral | Positive | Positive | Negative | Positive |
| **4** | I can't say the kitchen hygiene was anything I'd complain about.... | negation of a negative -> mildly positive | Positive | Negative | Negative | Negative | Positive |

In [17]:
```
# ---- quick tally: how many of the 5 tricky cases each approach got right ------
graded = ["Negative" if l == "Negative" else "Positive" for l in tricky["human_labe
for col in ["NB/LogReg", "BiLSTM", "DistilBERT", "LLM 0-shot", "LLM few-shot"]:
```

```
    hits = sum(a == b for a, b in zip(stress[col], graded))
    print(f"{col:13s}: {hits}/5   (neutral row is graded lenient — either polarity
print("\nThe neutral/ambiguous review has no gold polarity; we accept either label
```

```
NB/LogReg    : 2/5   (neutral row is graded lenient — either polarity accepted)
BiLSTM       : 2/5   (neutral row is graded lenient — either polarity accepted)
DistilBERT   : 2/5   (neutral row is graded lenient — either polarity accepted)
LLM 0-shot   : 4/5   (neutral row is graded lenient — either polarity accepted)
LLM few-shot : 4/5   (neutral row is graded lenient — either polarity accepted)

The neutral/ambiguous review has no gold polarity; we accept either label there.
```

**What the stress test shows.** The LLM (zero-shot and few-shot) leads at **4/5**; every trained model manages only **2/5**.

- **Negation / double negation / litotes** (rows 0 and 4) — "*not bad at all … some of the best*" and "*can't say the hygiene was anything I'd complain about*". TF-IDF + LogReg and the BiLSTM get **both wrong** (bag-of-words cannot bind "not"/"can't" to what it negates, and the BiLSTM never saw enough of these constructions). DistilBERT gets the double negation; only the LLM gets the litotes.
- **Sarcasm** (row 1) — "*two-hour delivery … stone cold. Fantastic service, truly.*" Interestingly the **lexical models get this right** — "cold", "two-hour delivery" outweigh "fantastic" — while **DistilBERT and the LLM are fooled** by the polite surface form. Nobody is reliable on sarcasm; the lexical models were essentially lucky on negative keywords.
- **Mixed sentiment** (row 2) — great taste, leaking packaging, 40 min late. DistilBERT and the LLM resolve it to Negative (matching the business definition of a dissatisfaction signal); NB/LogReg and the BiLSTM are swayed positive by "amazing" / "huge".
- **Neutral** (row 3) — no gold polarity; all models are forced into a binary guess, which is itself a limitation of a 2-class setup for this domain.

Net: aggregate F1 ranks the tuned linear model first among the trained approaches, but the stress test exposes that its wins are lexical — it collapses on exactly the negation and mixed-aspect cases that matter for "flag genuine dissatisfaction", where the LLM is materially more robust.

## Final Written Summary (Required)

*Your summary (~370 words):*

**High-traffic, cost-sensitive deployment (a food-delivery platform scanning every incoming review to flag restaurant-partner quality problems).** Deploy the **tuned TF-IDF + Logistic Regression** model. On the full 2,000-row test set it had the best F1 of any model trained and evaluated on that full set (0.928, vs 0.893 for the BiLSTM; DistilBERT's 0.907 is on a 600-row subset). It trains in seconds, serves in ~3 ms per review on commodity CPU, costs nothing per call, and is fully interpretable — every flag comes with the exact words that triggered it, which matters before you penalise a partner restaurant. The zero-shot LLM was more accurate (0.973) but ~185–400 ms per review plus a per-call API bill; at millions of

reviews per day that is not worth ~4 F1 points. A properly (GPU) full-fine-tuned DistilBERT is the upgrade path once accuracy, not cost, becomes the binding constraint.

**Low-volume, high-stakes use case (a small team triaging serious hygiene / foreign-object complaints).** Put an **LLM in the loop**: zero-shot prompting as the primary classifier with a human reviewing every case, optionally backed by a fine-tuned DistilBERT for on-prem redundancy. Volume is low, so latency and a few cents per 1,000 calls are irrelevant, and the LLM's strengths are exactly what this needs — it was the only approach to handle negation and litotes in the stress test (4/5 vs 2/5 for every trained model), and it can additionally name the failing aspect and draft an explanation for the reviewer. Where the complaint text cannot leave company infrastructure, run a local open model (as in Part E) or the fine-tuned transformer instead.

**One privacy / ethics consideration (Session 7 — Privacy & Ethics in NLP).** Restaurant and delivery reviews are personal data: free text routinely contains names, addresses, phone numbers, order history and health information ("I got food poisoning"), all linkable to an identifiable account. Sending it to a third-party LLM API exports that data outside the platform's control — it may be logged, retained, used to train the vendor's models, or processed under another jurisdiction, breaching GDPR / DPDP purpose-limitation and users' reasonable expectations. Mitigations: PII redaction before any external call, a data-processing agreement with training opt-out and short retention, or a self-hosted open model for anything sensitive — which is why the local model in Part E is a privacy design choice, not just a convenience.

# Submission Checklist

- ☑ All code cells completed and executed top-to-bottom without errors
- ☑ All 5 reflection questions answered (Q1–Q5)
- ☑ Comparison table (Part F) fully populated with real numbers
- ☑ Final written summary completed (250–400 words, both deployment scenarios + ethics)
- ☑ Notebook named `PS4_Group86_NLP_SentimentAnalysis_Assignment.ipynb`
- ☑ Group number, problem statement, and all four members' names/IDs filled in at the top
- ☑ No API keys or secrets in the notebook — Part E uses a local open-source model

**Academic Integrity:** AI assistants were used for boilerplate/scaffolding; the analysis, reflection answers and final summary reflect our own interpretation of the results produced by this notebook.